ROS2 기초편

#29 Depth 카메라와 OpenCV

CONTENTS

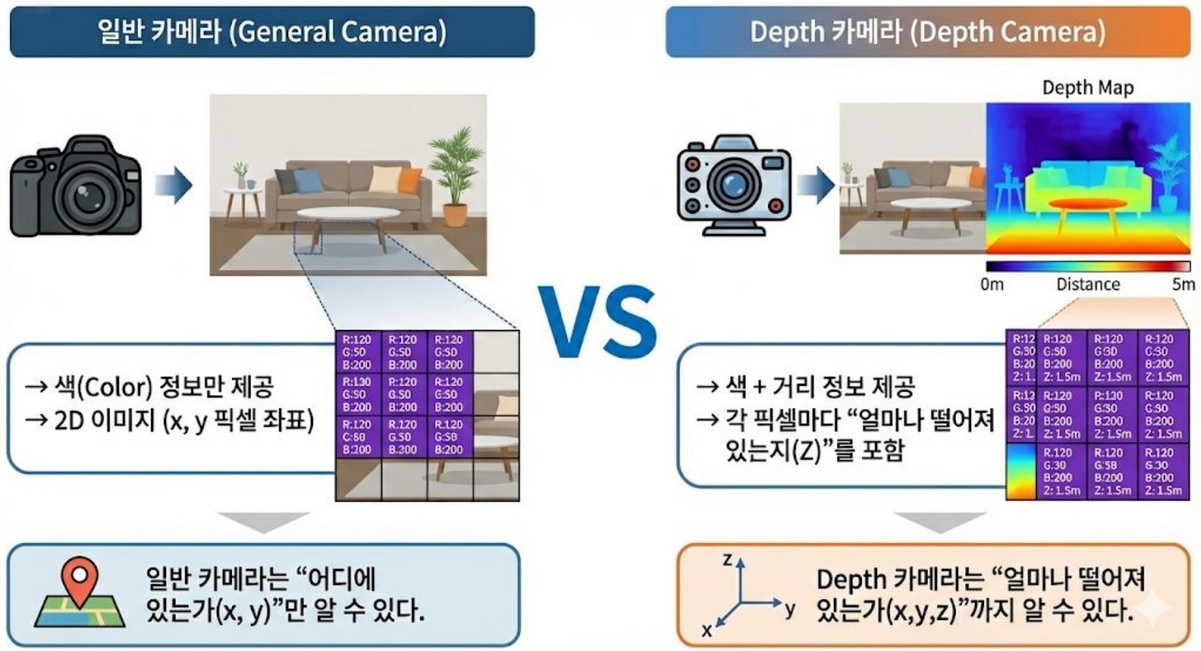
01 Depth 카메라란?

02 OpenCV란?

03 Depth 카메라 실습

일반 카메라 vs Depth 카메라

- 일반 카메라
  → 색(Color) 정보만 제공
  → 2D 이미지 (x, y 픽셀 좌표)
- Depth 카메라
  → 색 + 거리 정보 제공
  → 각 픽셀마다 "얼마나 떨어져 있는지(Z)"를 포함



Depth 카메라의 역할

- 물체까지의 거리 측정
- 3D 좌표 계산 (X, Y, Z)
- Pick & Place를 위한 위치 인식
- 작업 공간 안전 감지

Depth 카메라 패키지 설치

- RealSence 카메라 드라이버 설치
- PC에 카메라 연결 후 RealSense 카메라 노드를 실행하는 런치 파일을 실행

```
deeptree@robotdive:~$ sudo apt install ros-humble-realsense2-camera
```

```
deeptree@robotdive:~$ ros2 launch realsense2_camera rs_launch.py
[INFO] [launch]: All log files can be found below /home/deeptree/.ros/log/2026-02-23-14-00-38-478515-robotdive-59560
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [launch.user]: 🚀 Launching as Normal ROS Node
```

Depth 카메라 토픽 확인

- RealSence 카메라 노드가 실행된 상태에서 topic list 확인

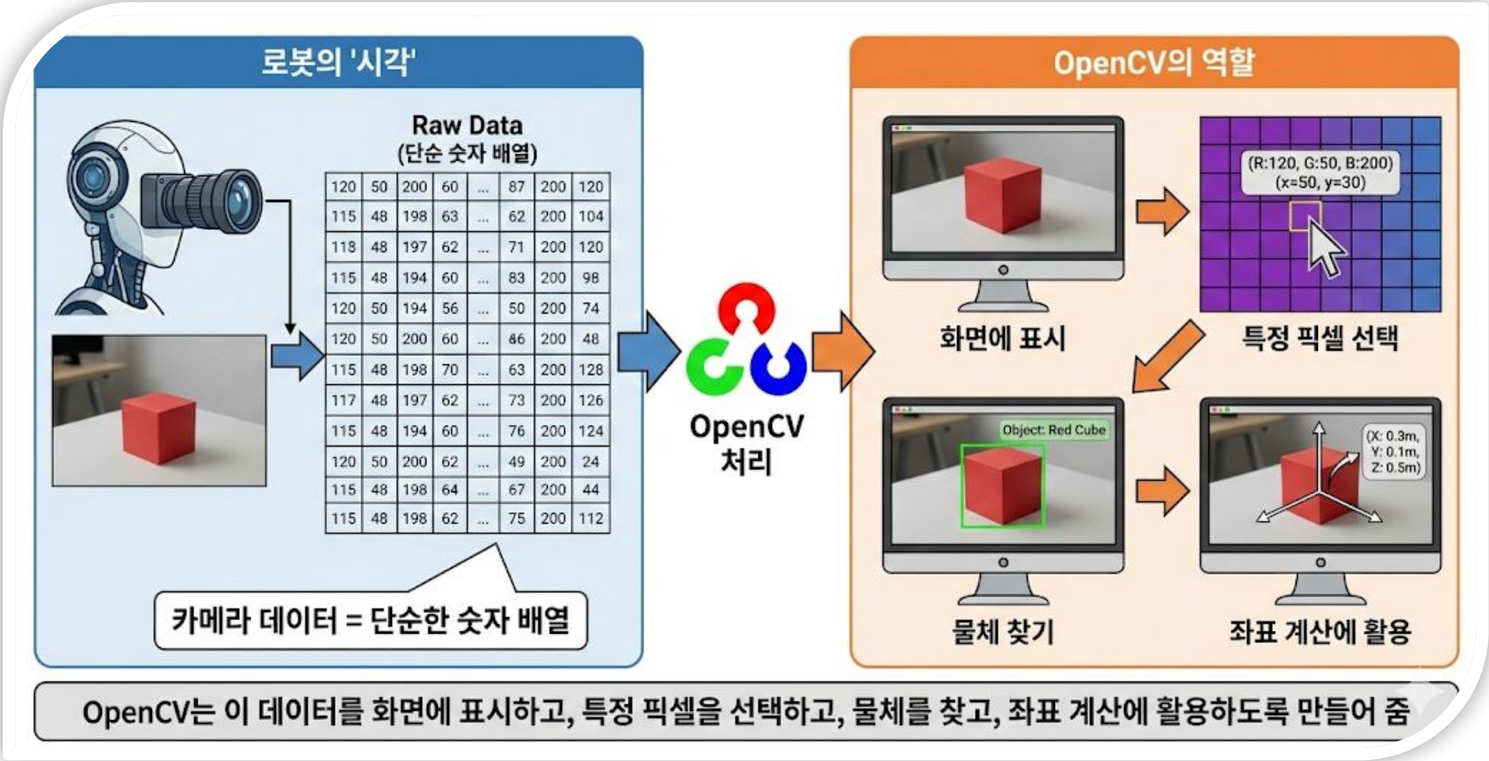
```
deeptree@robotdive:~$ ros2 topic list
/camera/camera/color/camera_info
/camera/camera/color/image_raw
/camera/camera/color/image_raw/compressed
/camera/camera/color/image_raw/compressedDepth
/camera/camera/color/image_raw/theora
/camera/camera/color/metadata
/camera/camera/depth/camera_info
/camera/camera/depth/image_rect_raw
/camera/camera/depth/image_rect_raw/compressed
/camera/camera/depth/image_rect_raw/compressedDepth
/camera/camera/depth/image_rect_raw/theora
/camera/camera/depth/metadata
/camera/camera/extrinsics/depth_to_color
/parameter_events
/rosout
/tf_static
```

OpenCV(Open Source Computer Vision Library)

- 이미지 및 영상 처리를 위한 라이브러리

로봇에서의 역할

- 로봇은 카메라로 환경을 보지만 카메라 데이터는 단순한 숫자 배열
- OpenCV는 이 데이터를 화면에 표시하고 특정 픽셀을 선택하고 물체를 찾고 좌표 계산에 활용하도록 만들어 줌



필요한 라이브러리 설치

```
sudo apt install ros-humble-cv-bridge ros-humble-image-transport
sudo apt install -y python3-opencv
pip3 install "numpy>=1.17.3,<1.25.0"
pip3 install "scipy>=1.7,<1.11"
```

```
deeptree@robotdive:~$ sudo apt install ros-humble-cv-bridge ros-humble-image-transport

deeptree@robotdive:~$ sudo apt install -y python3-opencv

deeptree@robotdive:~$ pip3 install "numpy>=1.17.3,<1.25.0"

deeptree@robotdive:~$ pip3 install "scipy>=1.7,<1.11"
```

실습용 Depth 카메라 패키지 생성

- depth_ws 워크스페이스와 src 폴더 생성 후 depth_pkg 패키지 생성

```
mkdir -p ~/depth_ws/src
cd ~/depth_ws/src
ros2 pkg create depth_pkg --build-type ament_python --dependencies rclpy sensor_msgs cv_bridge
```

```
deeptree@robotdive:~$ mkdir -p ~/depth_ws/src
deeptree@robotdive:~$ cd ~/depth_ws/src/
```

```
deeptree@robotdive:~/depth_ws/src$ ros2 pkg create depth_pkg --build-type ament_python --dependencies rclpy sensor_msgs cv_bridge
```

.bashrc 수정

- 터미널에서 nano로 .bashrc 마지막 줄에 setup.bash 추가

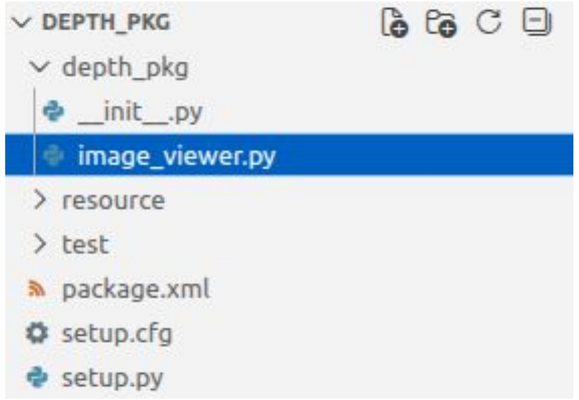
```
deeptree@robotdive:~/depth_ws/src$ nano ~/.bashrc
```

```
alias depth_ws="source ~/depth_ws/install/setup.bash; echo \"depth_ws is activated\""
```

image_viewer 노드 생성

- VS Code로 depth_pkg 열기
- depth_pkg 안에 image_viewer 노드 생성

```
deeptree@robotdive:~/depth_ws/src$ code depth_pkg/
```



image_viewer.py 작성

- Depth 카메라로 입력된 이미지 송출하는 노드

- COLOR_IMAGE_TOPIC = "/camera/camera/color/image_raw" :
- 카메라가 퍼블리시하는 토픽 이름을 저장

- COLOR_INFO_TOPIC = "/camera/camera/color/camera_info" :
- 코드에서 반복 입력을 줄이기 위한 상수 정의

depth_pkg > image_viewer.py > ImageViewerNode > __init__

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

from sensor_msgs.msg import Image, CameraInfo
from cv_bridge import CvBridge
import cv2


COLOR_IMAGE_TOPIC = "/camera/camera/color/image_raw"
COLOR_INFO_TOPIC  = "/camera/camera/color/camera_info"
```

image_viewer.py 작성

- ImageViewerNode : 노드 클래스 정의
- self.bridge = CvBridge() : ROS Image를 OpenCV 이미지로 바꾸기 위한 객체 생성
- fx, fy, cx, cy
  → depth 카메라의 내부 파라미터(intrinsic)를 저장할 변수. 처음엔 None
  - fx, fy :초점 거리 (CameraInfo에서 수신)
  - cx, cy : 주점, 이미지 중심 좌표 (CameraInfo에서 수신)
- /camera_info 토픽 구독
- 메시지가 오면 camera_info_callback() 실행

```python
class ImageViewerNode(Node):
    def __init__(self):
        super().__init__("image_viewer")

        self.bridge = CvBridge()
        self.camera_info = None

        self.create_subscription(
            CameraInfo,
            COLOR_INFO_TOPIC,
            self.camera_info_callback,
            10,
        )

        self.create_subscription(
            Image,
            COLOR_IMAGE_TOPIC,
            self.image_callback,
            10,
        )

        self.get_logger().info("depth_pkg image viewer started")

    def camera_info_callback(self, msg: CameraInfo):
        if self.camera_info is None:
            self.camera_info = msg
            fx = msg.k[0]
            fy = msg.k[4]
            cx = msg.k[2]
            cy = msg.k[5]

            self.get_logger().info(
                f"fx={fx:.2f}, fy={fy:.2f}, cx={cx:.2f}, cy={cy:.2f}"
            )

    def image_callback(self, msg: Image):
        frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")
        cv2.imshow("Color", frame)

        if cv2.waitKey(1) == 27:
            rclpy.shutdown()
```

image_viewer.py 작성

- /image_raw 구독
- 영상이 들어오면 image_callback() 실행
- camera_info_callback : 카메라 내부 파라미터 읽기
  - msg.k는 3×3 카메라 행렬 K를 1차원 배열로 펼친 것:
  - k[0] = fx, k[4] = fy, k[2] = cx, k[5] = cy
- image_callback : ROS Image 메시지를 OpenCV 이미지로 변환

```python
class ImageViewerNode(Node):
    def __init__(self):
        super().__init__("image_viewer")

        self.bridge = CvBridge()
        self.camera_info = None

        self.create_subscription(
            CameraInfo,
            COLOR_INFO_TOPIC,
            self.camera_info_callback,
            10,
        )

        self.create_subscription(
            Image,
            COLOR_IMAGE_TOPIC,
            self.image_callback,
            10,
        )

        self.get_logger().info("depth_pkg image viewer started")

    def camera_info_callback(self, msg: CameraInfo):
        if self.camera_info is None:
            self.camera_info = msg
            fx = msg.k[0]
            fy = msg.k[4]
            cx = msg.k[2]
            cy = msg.k[5]

            self.get_logger().info(
                f"fx={fx:.2f}, fy={fy:.2f}, cx={cx:.2f}, cy={cy:.2f}"
            )

    def image_callback(self, msg: Image):
        frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")
        cv2.imshow("Color", frame)

        if cv2.waitKey(1) == 27:
            rclpy.shutdown()
```

image_viewer.py 작성

- rclpy.init() → ROS2 초기화
- 노드 생성
- rclpy.spin(node) → 콜백 대기
- 종료 시 자원 정리

```python
def main(args=None):
    rclpy.init(args=args)
    node = ImageViewerNode()
    rclpy.spin(node)
    node.destroy_node()
    cv2.destroyAllWindows()
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

setup.py 수정

- entry_points에 image_viewer 추가

setup.py > ...

```python
from setuptools import find_packages, setup

package_name = 'depth_pkg'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='deeptree',
    maintainer_email='xxbb96@gmail.com',
    description='TODO: Package description',
    license='TODO: License declaration',
    extras_require={
        'test': [
            'pytest',
        ],
    },
    entry_points={
        'console_scripts': [
            'image_viewer = depth_pkg.image_viewer:main',
        ],
    },
)
```

Depth 카메라 실행

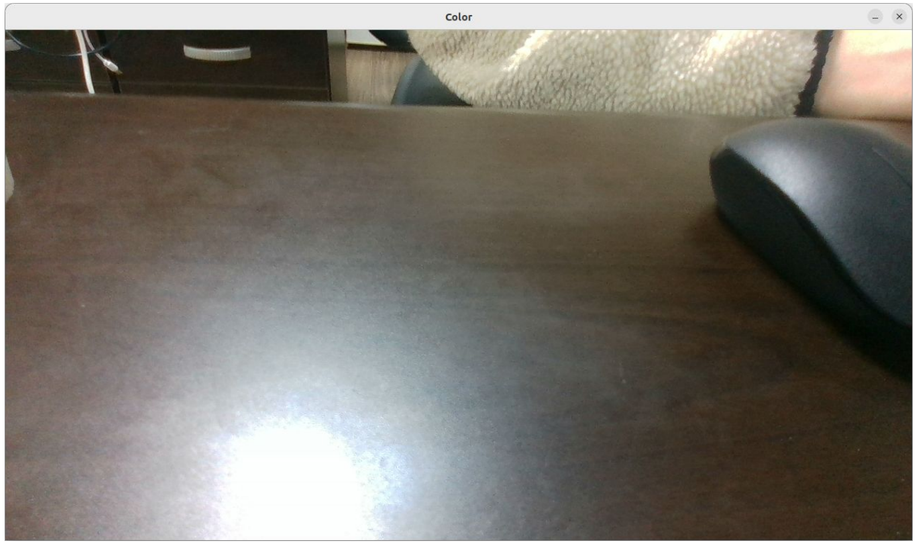
- 터미널을 열고 RealSence 카메라 노드 실행

```
deeptree@robotdive:~$ ros2 launch realsense2_camera rs_launch.py
```

image_viewer 노드 실행

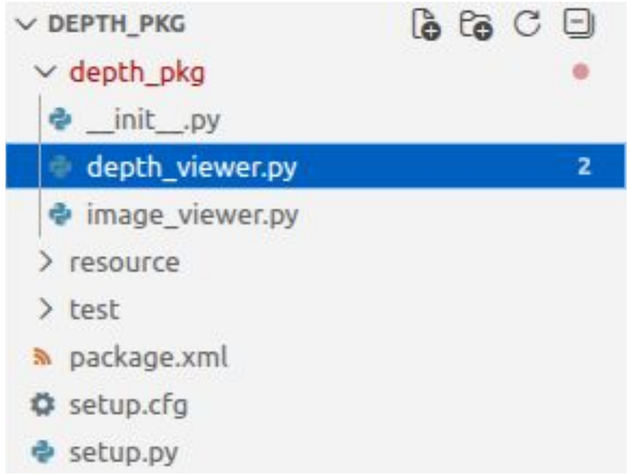
- 다른 터미널에서 빌드 후 image_viewer 실행

```
deeptree@robotdive:~/depth_ws/src$ cd ~/depth_ws/
deeptree@robotdive:~/depth_ws$ colcon build
Starting >>> depth_pkg
Finished <<< depth_pkg [0.57s]

Summary: 1 package finished [0.74s]
deeptree@robotdive:~/depth_ws$ depth_ws
depth_ws is activated
deeptree@robotdive:~/depth_ws$ ros2 run depth_pkg image_viewer
[INFO] [1771827987.141125079] [image_viewer]: depth_pkg image viewer started
[INFO] [1771827987.142269768] [image_viewer]: fx=904.81, fy=904.45, cx=639.16, cy=370.40
```



depth_viewer 노드 생성

- depth_pkg 안에 depth_viewer 노드 생성



depth_viewer.py 작성

- 화면 중앙의 픽셀값의 깊이를 출력하는 노드

- DEPTH_IMAGE_TOPIC = "/camera/camera/depth/image_rect_raw" :
- RealSense 드라이버가 퍼블리시하는 깊이 정보 토픽

depth_pkg > depth_viewer.py > DepthCenterViewer

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

from sensor_msgs.msg import Image, CameraInfo
from cv_bridge import CvBridge
import cv2

# === 토픽 이름 (ros2 topic list로 확인한 걸 그대로 넣기) ===
COLOR_IMAGE_TOPIC = "/camera/camera/color/image_raw"
DEPTH_IMAGE_TOPIC = "/camera/camera/depth/image_rect_raw"
CAMERA_INFO_TOPIC = "/camera/camera/depth/camera_info"
# ==========================================================
```

depth_viewer.py 작성

- super().__init__("depth_center_viewer")
  → 노드 이름을 "depth_center_viewer"로 등록
- self.bridge
  → ROS 이미지 ↔ OpenCV 이미지 변환용 CvBridge 객체
- fx, fy, cx, cy
  → depth 카메라의 내부 파라미터(intrinsic)를 저장할 변수. 처음엔 None
  - fx, fy :초점 거리 (CameraInfo에서 수신)
  - cx, cy : 주점, 이미지 중심 좌표 (CameraInfo에서 수신)
- depth_frame
  → 가장 최근에 수신한 depth 이미지를 저장
- print_counter
  → 로그를 매 프레임마다 찍지 않고, 일정 주기마다만 찍기 위해 사용하는 카운터

```python
class DepthCenterViewer(Node):
    """
    1) 컬러 이미지 구독 → 화면 중앙에 점 표시
    2) depth 이미지 구독 → 중앙 픽셀 depth 읽기
    3) depth용 camera_info 구독 → fx, fy, cx, cy 추출
    4) 중앙 픽셀 (X, Y, Z) 카메라 좌표 계산
    """

    def __init__(self):
        super().__init__("depth_center_viewer")

        self.bridge = CvBridge()

        # camera intrinsics (depth 기준)
        self.fx = None
        self.fy = None
        self.cx = None
        self.cy = None

        # 가장 최근 depth 프레임
        self.depth_frame = None

        # 로그 출력 간격 제어
        self.print_counter = 0
```

depth_viewer.py 작성

- CameraInfo (depth 카메라 정보)
  → 도착하면 camera_info_callback 호출
- Image (depth 이미지)
  → 도착하면 depth_image_callback 호출
- Image (컬러 이미지)
  → 도착하면 color_image_callback 호출

```python
# --- 구독 설정 ---
self.create_subscription(
    CameraInfo,
    CAMERA_INFO_TOPIC,
    self.camera_info_callback,
    10,
)

self.create_subscription(
    Image,
    DEPTH_IMAGE_TOPIC,
    self.depth_image_callback,
    10,
)

self.create_subscription(
    Image,
    COLOR_IMAGE_TOPIC,
    self.color_image_callback,
    10,
)

self.get_logger().info("depth_pkg: DepthCenterViewer node started")
```

depth_viewer.py 작성

- 카메라 정보 콜백
- depth 카메라의 내부 파라미터를 한 번만 저장
- if self.fx is None: 조건으로, 첫 호출에서만 값을 설정하고 로그를 출력
  → 이후 들어오는 CameraInfo는 무시(어차피 거의 안 변함)

```python
    def camera_info_callback(self, msg: CameraInfo):
        if self.fx is None:
            self.fx = msg.k[0]
            self.fy = msg.k[4]
            self.cx = msg.k[2]
            self.cy = msg.k[5]

            self.get_logger().info(
                f"[CameraInfo(depth)] fx={self.fx:.2f}, fy={self.fy:.2f}, "
                f"cx={self.cx:.2f}, cy={self.cy:.2f}"
            )
```

depth_viewer.py 작성

- 깊이 이미지 콜백

- depth 이미지를 OpenCV 형식으로 변환:
- desired_encoding="passthrough" → 원래 포맷 (16UC1, mm 단위)을 그대로 사용

- 변환한 이미지를 self.depth_frame에 저장하여 나중에 color 콜백에서 참조

- 첫 프레임에 대해서만 해상도 정보(w, h)를 로그로 출력해서 디버깅에 활용

```python
    def depth_image_callback(self, msg: Image):
        depth_img = self.bridge.imgmsg_to_cv2(msg, desired_encoding="passthrough")
        self.depth_frame = depth_img

        # 디버그용: 첫 프레임에서 한 번만 찍기
        h, w = depth_img.shape[:2]
        if self.print_counter == 0:
            self.get_logger().info(f"[Depth] first frame shape w={w}, h={h}")
```

depth_viewer.py 작성

- ROS Image → OpenCV 이미지로 변환
- 이미지 크기 구하고 중앙 좌표 (u, v) 계산
- depth 이미지와 카메라 내부 파라미터가 준비됐는지 확인, 준비 안 됐으면 파란 점 표시
- depth_raw : 중앙 픽셀의 depth 읽기

```python
def color_image_callback(self, msg: Image):
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")

    h_c, w_c, _ = frame.shape
    u = w_c // 2
    v = h_c // 2

    if (
        self.depth_frame is not None
        and self.fx is not None
        and self.fy is not None
    ):
        # 해상도 848x480으로 같다고 가정
        h_d, w_d = self.depth_frame.shape[:2]

        # 방어적 체크
        if (h_c, w_c) != (h_d, w_d):
            # 그래도 혹시 다르면 최소한 범위 안에서만 인덱싱
            u_depth = min(u, w_d - 1)
            v_depth = min(v, h_d - 1)
        else:
            u_depth = u
            v_depth = v

        depth_raw = self.depth_frame[v_depth, u_depth]

        if depth_raw > 0:
            Z = float(depth_raw) / 1000.0  # mm → m
            X = (u_depth - self.cx) * Z / self.fx
            Y = (v_depth - self.cy) * Z / self.fy

            self.print_counter += 1
            if self.print_counter % 30 == 0:
                self.get_logger().info(
                    f"Center pixel (u={u_depth}, v={v_depth}), "
                    f"depth_raw={depth_raw} -> "
                    f"X={X:.3f} m, Y={Y:.3f} m, Z={Z:.3f} m"
                )

            # 유효 depth → 빨간 점
            cv2.circle(frame, (u, v), 6, (0, 0, 255), -1)
        else:
            # depth=0 → 노란 점
            cv2.circle(frame, (u, v), 6, (0, 255, 255), -1)
    else:
        # depth/camera_info 준비 전 → 파란 점
        cv2.circle(frame, (u, v), 6, (255, 0, 0), -1)

    cv2.imshow("Color (center pixel depth)", frame)
    key = cv2.waitKey(1) & 0xFF
    if key == 27:
        rclpy.shutdown()
```

depth_viewer.py 작성

- Z: 카메라와의 거리
- X, Y: 카메라 좌표계 기준 위치
  → 중앙 픽셀의 실제 3D 위치 계산

- 빨간 점 → 유효한 depth
- 노란 점 → depth=0
- 파란 점 → 아직 준비 안 됨

```python
def color_image_callback(self, msg: Image):
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")

    h_c, w_c, _ = frame.shape
    u = w_c // 2
    v = h_c // 2

    if (
        self.depth_frame is not None
        and self.fx is not None
        and self.fy is not None
    ):
        # 해상도 848x480으로 같다고 가정
        h_d, w_d = self.depth_frame.shape[:2]

        # 방어적 체크
        if (h_c, w_c) != (h_d, w_d):
            # 그래도 혹시 다르면 최소한 범위 안에서만 인덱싱
            u_depth = min(u, w_d - 1)
            v_depth = min(v, h_d - 1)
        else:
            u_depth = u
            v_depth = v

        depth_raw = self.depth_frame[v_depth, u_depth]

        if depth_raw > 0:
            Z = float(depth_raw) / 1000.0  # mm → m
            X = (u_depth - self.cx) * Z / self.fx
            Y = (v_depth - self.cy) * Z / self.fy

            self.print_counter += 1
            if self.print_counter % 30 == 0:
                self.get_logger().info(
                    f"Center pixel (u={u_depth}, v={v_depth}), "
                    f"depth_raw={depth_raw} -> "
                    f"X={X:.3f} m, Y={Y:.3f} m, Z={Z:.3f} m"
                )

            # 유효 depth → 빨간 점
            cv2.circle(frame, (u, v), 6, (0, 0, 255), -1)
        else:
            # depth=0 → 노란 점
            cv2.circle(frame, (u, v), 6, (0, 255, 255), -1)
    else:
        # depth/camera_info 준비 전 → 파란 점
        cv2.circle(frame, (u, v), 6, (255, 0, 0), -1)

    cv2.imshow("Color (center pixel depth)", frame)
    key = cv2.waitKey(1) & 0xFF
    if key == 27:
        rclpy.shutdown()
```

depth_viewer.py 작성

- rclpy.init() → ROS2 초기화
- DepthCenterViewer 노드 생성
- rclpy.spin(node) → 콜백들을 돌면서 토픽 수신 대기.
- Ctrl+C 또는 ESC로 종료되면 노드 삭제, OpenCV 창 닫기, ROS2 종료

```python
def main(args=None):
    rclpy.init(args=args)
    node = DepthCenterViewer()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    cv2.destroyAllWindows()
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

setup.py 수정

- entry_points에 depth_viewer 추가

```python
from setuptools import find_packages, setup

package_name = 'depth_pkg'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='deeptree',
    maintainer_email='xxbb96@gmail.com',
    description='TODO: Package description',
    license='TODO: License declaration',
    extras_require={
        'test': [
            'pytest',
        ],
    },
    entry_points={
        'console_scripts': [
            'image_viewer = depth_pkg.image_viewer:main',
            'depth_viewer = depth_pkg.depth_viewer:main',
        ],
    },
)
```

Depth 카메라 실행

- 터미널을 열고 RealSence 카메라 노드 실행

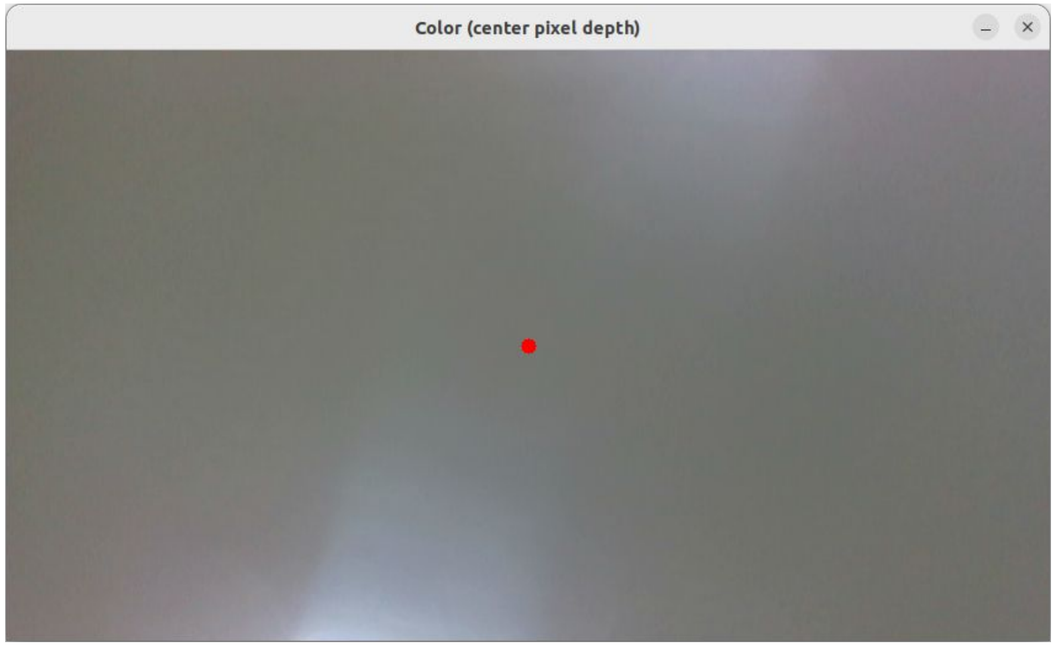
```
ros2 launch realsense2_camera rs_launch.py depth_module.depth_profile:=848x480x30 rgb_camera.color_profile:=848x480x30
```

```
deeptree@robotdive:~$ ros2 launch realsense2_camera rs_launch.py depth_module.depth_profile:=848x480x30 rgb_camera.color_profile:=848x480x30
[INFO] [launch]: All log files can be found below /home/deeptree/.ros/log/2026-02-23-16-44-04-714209-robotdive-77366
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [launch.user]: 🚀 Launching as Normal ROS Node
```

depth_viewer 노드 실행

- 다른 터미널에서 빌드 후 depth_viewer 실행
- Depth 카메라를 이동시키면서 가운데 위치한 물체의 깊이 확인
- u, v : 픽셀 좌표 (이미지 좌표계)
- depth_raw : 원시 깊이값 (거리, mm 단위)

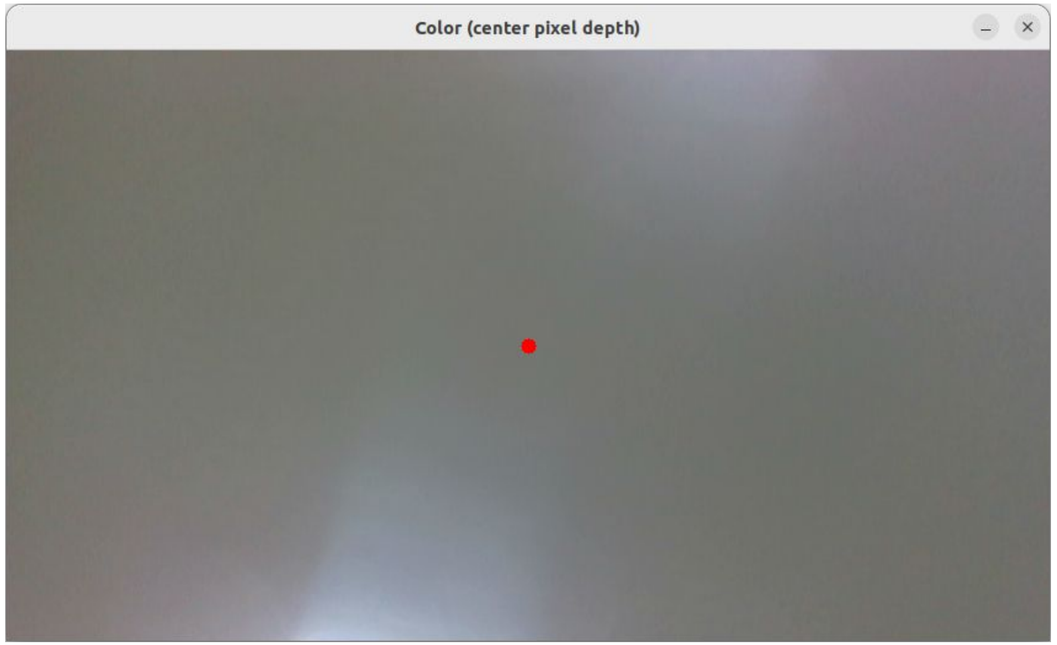
```
deeptree@robotdive:~/depth_ws$ colcon build
Starting >>> depth_pkg
Finished <<< depth_pkg [0.57s]

Summary: 1 package finished [0.74s]
deeptree@robotdive:~/depth_ws$ depth_ws
depth_ws is activated
deeptree@robotdive:~/depth_ws$ ros2 run depth_pkg depth_viewer

[INFO] [1771832483.727068222] [depth_center_viewer]: Center pixel (u=424, v=240), depth_raw=1483 -> X=-0.016 m, Y=0.027 m, Z=1.483 m
[INFO] [1771832484.728584626] [depth_center_viewer]: Center pixel (u=424, v=240), depth_raw=1490 -> X=-0.016 m, Y=0.027 m, Z=1.490 m
[INFO] [1771832485.729604775] [depth_center_viewer]: Center pixel (u=424, v=240), depth_raw=1461 -> X=-0.015 m, Y=0.026 m, Z=1.461 m
[INFO] [1771832486.729486037] [depth_center_viewer]: Center pixel (u=424, v=240), depth_raw=1434 -> X=-0.015 m, Y=0.026 m, Z=1.434 m
```



depth_viewer 노드 실행

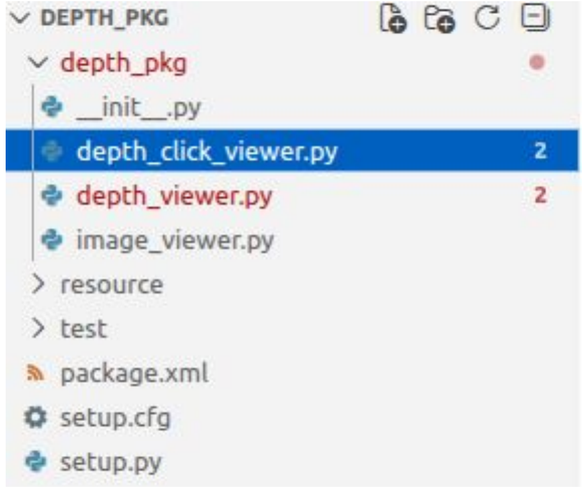
- u, v, depth_raw 세 값이 모두 있으면 카메라 좌표계 기준의 실제 3D 위치를 계산 가능
  - Z = depth_raw / 1000.0 # mm → m
  - X = (u - cx) * Z / fx
  - Y = (v - cy) * Z / fy

```
deeptree@robotdive:~/depth_ws$ colcon build
Starting >>> depth_pkg
Finished <<< depth_pkg [0.57s]

Summary: 1 package finished [0.74s]
deeptree@robotdive:~/depth_ws$ depth_ws
depth_ws is activated
deeptree@robotdive:~/depth_ws$ ros2 run depth_pkg depth_viewer

[INFO] [1771832483.727068222] [depth_center_viewer]: Center pixel (u=424, v=240), depth_raw=1483 -> X=-0.016 m, Y=0.027 m, Z=1.483 m
[INFO] [1771832484.728584626] [depth_center_viewer]: Center pixel (u=424, v=240), depth_raw=1490 -> X=-0.016 m, Y=0.027 m, Z=1.490 m
[INFO] [1771832485.729604775] [depth_center_viewer]: Center pixel (u=424, v=240), depth_raw=1461 -> X=-0.015 m, Y=0.026 m, Z=1.461 m
[INFO] [1771832486.729486037] [depth_center_viewer]: Center pixel (u=424, v=240), depth_raw=1434 -> X=-0.015 m, Y=0.026 m, Z=1.434 m
```



depth_click_viewer 노드 생성

- depth_pkg 안에 depth_click_viewer 노드 생성



depth_click_viewer.py 작성

- 화면에서 클릭한 위치의 깊이를 출력하는 노드

- g_click_u = None
  g_click_v = None
- → 마우스로 클릭한 픽셀 좌표를 저장하는 전역 변수

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

from sensor_msgs.msg import Image, CameraInfo
from cv_bridge import CvBridge
import cv2
import numpy as np

# === 토픽 이름 (ros2 topic list로 확인한 걸 그대로 넣기) ===
COLOR_IMAGE_TOPIC = "/camera/camera/color/image_raw"
DEPTH_IMAGE_TOPIC = "/camera/camera/depth/image_rect_raw"
CAMERA_INFO_TOPIC = "/camera/camera/depth/camera_info"
# ==========================================================

WINDOW_NAME = "Color (click pixel depth)"

# 마우스 클릭 좌표를 저장할 전역 변수
g_click_u = None
g_click_v = None
```

depth_click_viewer.py 작성

- 마우스 왼쪽 버튼 클릭 시
- 클릭한 픽셀 좌표 (u, v) 저장
- 이후 depth 계산에 사용

```python
def mouse_callback(event, x, y, flags, param):
    """
    마우스로 클릭한 픽셀 좌표(u, v)를 전역 변수에 저장.
    """
    global g_click_u, g_click_v
    if event == cv2.EVENT_LBUTTONDOWN:
        g_click_u = x
        g_click_v = y
        print(f"[Mouse] 클릭한 픽셀: u={g_click_u}, v={g_click_v}")
```

depth_click_viewer.py 작성

- namedWindow : 영상 창 생성
- setMouseCallback : 마우스 클릭 이벤트 연결

```python
class DepthClickViewer(Node):
    """
    1) 컬러 이미지 구독 → 클릭한 점(없으면 화면 중앙)에 동그라미 표시
    2) depth 이미지 구독 → 해당 픽셀의 depth 읽기
    3) depth용 camera_info 구독 → fx, fy, cx, cy 추출
    4) 클릭한 픽셀 (X, Y, Z) 카메라 좌표 계산
    """

    def __init__(self):
        super().__init__("depth_click_viewer")

        self.bridge = CvBridge()

        # camera intrinsics (depth 기준)
        self.fx = None
        self.fy = None
        self.cx = None
        self.cy = None

        self.depth_frame = None  # 최신 depth 이미지 (numpy)
        self.print_counter = 0   # 로그 출력 주기 조절용

        # --- Subscriber 설정 ---
        self.color_sub = self.create_subscription(
            Image,
            COLOR_IMAGE_TOPIC,
            self.color_image_callback,
            10,
        )
        self.depth_sub = self.create_subscription(
            Image,
            DEPTH_IMAGE_TOPIC,
            self.depth_image_callback,
            10,
        )
        self.camera_info_sub = self.create_subscription(
            CameraInfo,
            CAMERA_INFO_TOPIC,
            self.camera_info_callback,
            10,
        )

        # OpenCV 창 및 마우스 콜백 등록
        cv2.namedWindow(WINDOW_NAME)
        cv2.setMouseCallback(WINDOW_NAME, mouse_callback)

        self.get_logger().info("DepthClickViewer 노드가 시작되었습니다.")
```

depth_click_viewer.py 작성

- camera_info_callback
  - 카메라 내부 파라미터 추출
  - 한 번 받고 나면 구독 종료
- depth_image_callback
  - depth 이미지 수신
  - numpy 배열로 변환 후 저장

```python
    def camera_info_callback(self, msg: CameraInfo):
        # K = [fx, 0, cx,
        #      0, fy, cy,
        #      0,  0,  1]
        self.fx = msg.k[0]
        self.fy = msg.k[4]
        self.cx = msg.k[2]
        self.cy = msg.k[5]

        # 한 번만 로깅해도 충분하므로, 다른 값에서 None → 값으로 바뀌는 시점에만 출력
        self.camera_info_sub.destroy()  # 더 이상 받을 필요 없으면 끊어도 됨
        self.get_logger().info(
            f"Camera intrinsics 수신: fx={self.fx:.2f}, fy={self.fy:.2f}, "
            f"cx={self.cx:.2f}, cy={self.cy:.2f}"
        )

    def depth_image_callback(self, msg: Image):
        # 16UC1(depth in mm) 가정
        depth_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding="passthrough")
        self.depth_frame = depth_image
```

depth_click_viewer.py 작성

- bridge.imgmsg_to_cv2 : ROS 이미지 → OpenCV 이미지
- 사용할 픽셀 결정
  - 클릭한 점이 있으면 그 좌표 사용
  - 없으면 화면 중앙 사용
- depth_raw : 해당 픽셀의 거리(mm) 읽기
- 클릭한 픽셀의 카메라 좌표계 기준 (X, Y, Z)
- 점 색상 표시
  - 빨강 → 유효 depth
  - 노랑 → depth=0
  - 파랑 → 아직 준비 안 됨

```python
def color_image_callback(self, msg: Image):
    global g_click_u, g_click_v

    # BGR8 컬러 이미지
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")
    h_c, w_c, _ = frame.shape

    # 클릭한 점이 있으면 그 점 사용, 없으면 화면 중앙 사용
    if g_click_u is not None and g_click_v is not None:
        u = int(np.clip(g_click_u, 0, w_c - 1))
        v = int(np.clip(g_click_v, 0, h_c - 1))
        clicked = True
    else:
        u = w_c // 2
        v = h_c // 2
        clicked = False

    # depth / camera_info가 준비된 상태인지 확인
    if (
        self.depth_frame is not None
        and self.fx is not None
        and self.fy is not None
    ):
        h_d, w_d = self.depth_frame.shape[:2]

        # 컬러/깊이 해상도가 다를 수 있으므로 방어적 인덱싱
        u_depth = int(np.clip(u, 0, w_d - 1))
        v_depth = int(np.clip(v, 0, h_d - 1))

        depth_raw = self.depth_frame[v_depth, u_depth]

        if depth_raw > 0:
            Z = float(depth_raw) / 1000.0  # mm → m
            X = (u_depth - self.cx) * Z / self.fx
            Y = (v_depth - self.cy) * Z / self.fy

            self.print_counter += 1
            # 30프레임마다 한 번씩만 출력 (너무 많이 찍히지 않도록)
            if self.print_counter % 30 == 0:
                where = "클릭한 픽셀" if clicked else "중심 픽셀"
                self.get_logger().info(
                    f"{where} (u={u_depth}, v={v_depth}), "
                    f"depth_raw={depth_raw} -> "
                    f"X={X:.3f} m, Y={Y:.3f} m, Z={Z:.3f} m"
                )

            # 유효 depth → 빨간 점 (선택점이라는 의미로)
            color = (0, 0, 255)
        else:
            # depth=0 → 노란 점
            color = (0, 255, 255)
    else:
        # depth/camera_info 준비 전 → 파란 점
        color = (255, 0, 0)
```

depth_click_viewer.py 작성

- 어디를 보고 계산 중인지 시각적으로 표시하고, 사용자가 ESC로 프로그램을 종료할 수 있게 처리

```
# 선택된 픽셀(또는 중앙 픽셀)에 동그라미 표시
cv2.circle(frame, (u, v), 6, color, -1)

# 안내 텍스트
text = "Click pixel to see depth & XYZ"
cv2.putText(
    frame,
    text,
    (10, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (255, 255, 255),
    2,
    cv2.LINE_AA,
)

cv2.imshow(WINDOW_NAME, frame)
key = cv2.waitKey(1) & 0xFF
if key == 27:  # ESC
    rclpy.shutdown()
```

depth_click_viewer.py 작성

- ROS2 초기화
- 노드 생성
- spin()으로 콜백 대기
- 종료 시 자원 정리

```
def main(args=None):
    rclpy.init(args=args)
    node = DepthClickViewer()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    cv2.destroyAllWindows()
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

setup.py 수정

- entry_points에 depth_click_viewer 추가

```python
from setuptools import find_packages, setup

package_name = 'depth_pkg'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='deeptree',
    maintainer_email='xxbb96@gmail.com',
    description='TODO: Package description',
    license='TODO: License declaration',
    extras_require={
        'test': [
            'pytest',
        ],
    },
    entry_points={
        'console_scripts': [
            'image_viewer = depth_pkg.image_viewer:main',
            'depth_viewer = depth_pkg.depth_viewer:main',
            'depth_click_viewer = depth_pkg.depth_click_viewer:main',
        ],
    },
)
```

Depth 카메라 실행

- 터미널을 열고 RealSence 카메라 노드 실행

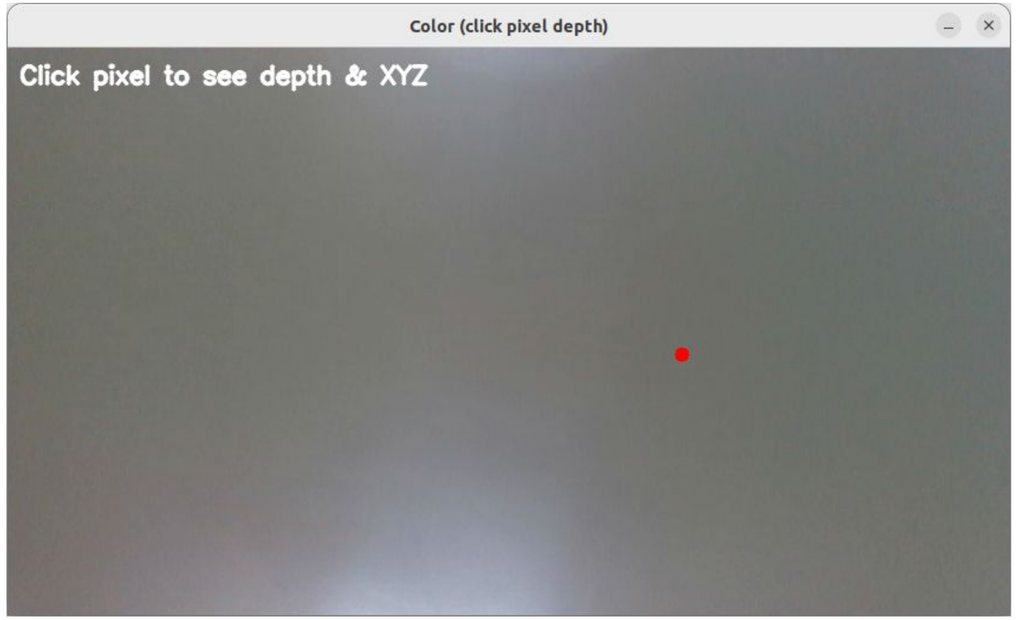
```
ros2 launch realsense2_camera rs_launch.py depth_module.depth_profile:=848x480x30 rgb_camera.color_profile:=848x480x30
```

```
deeptree@robotdive:~$ ros2 launch realsense2_camera rs_launch.py depth_module.depth_profile:=848x480x30 rgb_camera.color_profile:=848x480x30
[INFO] [launch]: All log files can be found below /home/deeptree/.ros/log/2026-02-23-16-44-04-714209-robotdive-77366
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [launch.user]: 🚀 Launching as Normal ROS Node
```

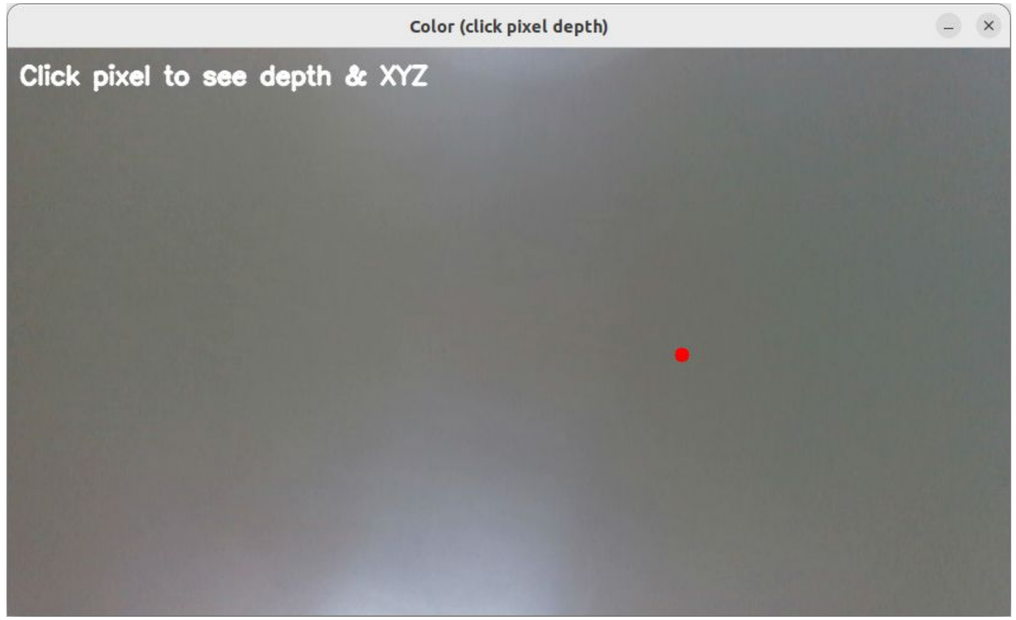
depth_click_viewer 노드 실행

- 다른 터미널에서 빌드 후 depth_click_viewer 실행
- Depth 카메라로 촬영중인 화면을 클릭해 그 픽셀 정보 확인
- u, v : 픽셀 좌표 (이미지 좌표계)
- depth_raw : 원시 깊이값 (거리, mm 단위)

```
deeptree@robotdive:~/depth_ws$ colcon build
Starting >>> depth_pkg
Finished <<< depth_pkg [0.57s]

Summary: 1 package finished [0.73s]
deeptree@robotdive:~/depth_ws$ depth_ws
depth_ws is activated
deeptree@robotdive:~/depth_ws$ ros2 run depth_pkg depth_click_viewer
[INFO] [1771838244.786897272] [depth_click_viewer]: DepthClickViewer 노드가 시작되었습니다.
[INFO] [1771838244.799361351] [depth_click_viewer]: Camera intrinsics 수신: fx=429.04, fy=429.04, cx=428.53, cy=232.22

[Mouse] 클릭한 픽셀: u=570, v=259
[INFO] [1771838435.464744130] [depth_click_viewer]: 클릭한 픽셀 (u=570, v=259), depth_raw=1440 -> X=0.475 m, Y=0.090 m, Z=1.440 m
[INFO] [1771838436.467338863] [depth_click_viewer]: 클릭한 픽셀 (u=570, v=259), depth_raw=1458 -> X=0.481 m, Y=0.091 m, Z=1.458 m
```



세 값을 이용한 3D 좌표 변환

- u, v, depth_raw 세 값이 모두 있으면 카메라 좌표계 기준의 실제 3D 위치를 계산 가능
  - Z = depth_raw / 1000.0 # mm → m
  - X = (u - cx) * Z / fx
  - Y = (v - cy) * Z / fy

```
deeptree@robotdive:~/depth_ws$ colcon build
Starting >>> depth_pkg
Finished <<< depth_pkg [0.57s]

Summary: 1 package finished [0.73s]
deeptree@robotdive:~/depth_ws$ depth_ws
depth_ws is activated
deeptree@robotdive:~/depth_ws$ ros2 run depth_pkg depth_click_viewer
[INFO] [1771838244.786897272] [depth_click_viewer]: DepthClickViewer 노드가 시작되었습니다.
[INFO] [1771838244.799361351] [depth_click_viewer]: Camera intrinsics 수신: fx=429.04, fy=429.04, cx=428.53, cy=232.22

[Mouse] 클릭한 픽셀: u=570, v=259
[INFO] [1771838435.464744130] [depth_click_viewer]: 클릭한 픽셀 (u=570, v=259), depth_raw=1440 -> X=0.475 m, Y=0.090 m, Z=1.440 m
[INFO] [1771838436.467338863] [depth_click_viewer]: 클릭한 픽셀 (u=570, v=259), depth_raw=1458 -> X=0.481 m, Y=0.091 m, Z=1.458 m
```